目录

1 基于 CNN 的校园课堂智能考勤系统的设计与实现 4
1.1 中文摘要 5
1.2 Abstract 5
1.3 目录 6

2 第一章绪论 8
2.1 1.1 研究背景与意义 8
2.1.1 1.1.1 传统考勤方式的局限性 8
2.1.2 1.1.2 人脸识别技术的发展与应用前景 8
2.1.3 1.1.3 研究意义 9
2.2 1.2 国内外研究现状 9
2.2.1 1.2.1 人脸识别技术研究现状 9
2.2.2 1.2.2 智能考勤系统研究现状 10
2.2.3 1.2.3 现有研究的不足 10
2.3 1.3 研究内容与目标 10
2.3.1 1.3.1 研究内容 10
2.3.2 1.3.2 性能目标 11
2.4 1.4 论文结构安排 11

3 第二章相关技术综述 12
3.1 2.1 卷积神经网络基础 12
3.1.1 2.1.1 CNN 的基本结构 12
3.1.2 2.1.2 VGG16 网络结构 13
3.1.3 2.1.3 ResNet50 网络结构与残差连接 13
3.2 2.2 迁移学习 14
3.2.1 2.2.1 迁移学习原理 14
3.2.2 2.2.2 两阶段微调策略 14
3.3 2.3 人脸检测：MTCNN 15
3.3.1 2.3.1 MTCNN 基本原理 15
3.3.2 2.3.2 相比传统方法的优势 16
3.4 2.4 特征匹配：余弦相似度与双阈值机制 16
3.4.1 2.4.1 L2 归一化与余弦相似度 16
3.4.2 2.4.2 双阈值判断机制 16

3.5 2.5 Web 技术栈 17
3.5.1 2.5.1 Flask 框架 17
3.5.2 2.5.2 Socket.IO 实时通信 17
3.5.3 2.5.3 Vue 3 与 Pinia 17
3.5.4 2.5.4 JWT 身份认证 18

4 第三章需求分析 18
4.1 3.1 系统概述 18
4.2 3.2 用户角色与权限分析 18
4.2.1 3.2.1 用户角色定义 18
4.2.2 3.2.2 权限矩阵 19
4.3 3.3 功能需求 19
4.3.1 3.3.1 用户管理模块（F1） 19
4.3.2 3.3.2 学生档案模块（F2） 19
4.3.3 3.3.3 人脸数据管理模块（F3） 20
4.3.4 3.3.4 考勤任务模块（F4）——核心功能 20
4.3.5 3.3.5 考勤记录模块（F5） 20
4.3.6 3.3.6 报表统计模块（F6） 21
4.4 3.4 非功能需求 21
4.4.1 3.4.1 性能需求 21
4.4.2 3.4.2 安全需求 21
4.4.3 3.4.3 可用性需求 21
4.5 3.5 系统用例分析 22

5 第四章系统设计 23
5.1 4.1 系统总体架构 23
5.1.1 4.1.1 三层架构设计 23
5.1.2 4.1.2 技术选型说明 24
5.2 4.2 数据库设计 24
5.2.1 4.2.1 E-R 图 24
5.2.2 4.2.2 核心数据表设计 24
5.2.3 4.2.3 索引设计 26
5.3 4.3 AI 模块设计 26
5.3.1 4.3.1 模块整体架构 26
5.3.2 4.3.2 推理流程设计 26

5.3.3 4.3.3 训练模块设计 27
5.4 4.4 后端接口设计 28
5.4.1 4.4.1 统一响应格式 28
5.4.2 4.4.2 核心接口列表 28
5.4.3 4.4.3 JWT 权限控制实现 29
5.5 4.5 前端架构设计 30
5.5.1 4.5.1 路由与权限守卫 30
5.5.2 4.5.2 核心组件结构 30
5.5.3 4.5.3 实时考勤前端状态机 30

6 第五章系统实现 31
6.1 5.1 后端核心实现 31
6.1.1 5.1.1 项目结构与蓝图划分 31
6.1.2 5.1.2 应用工厂与扩展注册 32
6.1.3 5.1.3 JWT 权限控制实现 33
6.1.4 5.1.4 考勤任务 API 实现 34
6.2 5.2 WebSocket 实时识别实现 35
6.2.1 5.2.1 服务端帧处理逻辑 35
6.2.2 5.2.2 防重复出勤处理 36
6.3 5.3 AI 推理模块实现 37
6.3.1 5.3.1 ModelManager 单例设计 37
6.3.2 5.3.2 特征提取：512 维 Embedding 37
6.3.3 5.3.3 特征库匹配与双阈值判断 38
6.4 5.4 前端核心实现 39
6.4.1 5.4.1 全局请求封装与错误处理 39
6.4.2 5.4.2 实时考勤组件实现 40
6.4.3 5.4.3 角色自适应菜单实现 41
6.5 5.5 本章小结 42

7 第六章系统测试与实验 42
7.1 6.1 功能测试 42
7.1.1 6.1.1 测试环境 42
7.1.2 6.1.2 功能测试用例 42
7.1.3 6.1.3 边界条件测试 43
7.2 6.2 AI 模型实验 44

7.2.1 6.2.1 实验数据集 44
7.2.2 6.2.2 训练配置 44
7.2.3 6.2.3 训练过程分析 45
7.2.4 6.2.4 测试集评估结果 46
7.3 6.3 VGG16 与 ResNet50 架构对比分析 47
7.3.1 6.3.1 架构参数对比 47
7.3.2 6.3.2 训练过程对比 48
7.3.3 6.3.3 测试集性能指标实测对比 48
7.3.4 6.3.4 模型选型结论 49
7.4 6.4 系统性能测试 50
7.4.1 6.4.1 接口响应时间 50
7.4.2 6.4.2 WebSocket 并发测试 50
7.5 6.5 本章小结 50

8 第七章总结与展望 51
8.1 7.1 工作总结 51
8.2 7.2 不足与局限 52
8.3 7.3 未来展望 52

9 参考文献 53

1 基于 CNN 的校园课堂智能考勤系统的设计与实现

题目：基于 CNN 的校园课堂智能考勤系统的设计与实现

学生姓名：苏东煌

学号：2022214380

指导教师：张晓霞

学院：计算机科学与技术学院（示范性软件学院）

年份：2026 年

1.1 中文摘要

在高校日常教学管理中，考勤是一项看似琐碎、实则颇为耗时的工作。传统的人工点名方式在班级规模较大时往往要花掉十几分钟，而签到表和二维码这类方案又很容易被学生钻空子代签，考勤数据的可信度存疑。本文的出发点，就是尝试用人脸识别技术来解决这个问题。

整个系统采用前后端分离的设计，后端用 Flask 搭建 RESTful 接口并通过 Flask-SocketIO 提供实时推送，前端用 Vue 3 写了一套包含多角色权限管理的 Web 界面。AI 识别部分先用 MTCNN 做人脸检测和对齐，再用 CNN 模型提取 512 维特征向量，最终通过余弦相似度来判断是谁。为了在实际使用中更灵活一些，设计了双阈值机制：相似度高于 0.75 的记为出勤，介于 0.50 到 0.75 之间的标为待核实交给老师复核，低于 0.50 则视为非注册学生忽略。这样可以把误识和漏识的问题都控制在相对可接受的范围内。

主干网络方面，本文分别对 VGG16 和 ResNet50 做了迁移学习实验并进行了对比。VGG16 在实验中对原始全连接头进行了全局平均池化（GAP）改造，完整训练 30 轮后验证集准确率达到 74.67%、测试集 Top-1 准确率 71.25%、AUC-ROC 98.22%，相比旧 FC 头版本大幅提升；但其卷积主干（约 1.38 亿参数，是 ResNet50 的 5 倍多）带来的 CPU 推理延迟达 405ms，超出实时性目标（≤200ms）一倍，为结构性约束。ResNet50 的结果更理想：验证集 Top-1 准确率 86.30%，AUC-ROC 99.30%，FAR 1.17%，CPU 单帧推理延迟 176.48ms，各项关键指标均达到任务书要求，因此最终选用 ResNet50 作为部署模型。

关键词：卷积神经网络；人脸识别；迁移学习；考勤系统；ResNet50；VGG16

1.2 Abstract

Taking attendance in a university classroom sounds trivial, but manual roll-call routinely eats 10–15 minutes per session, and sign-in sheets are easily forged. This paper describes an intelligent classroom attendance system built around face recognition, attempting to address both the time cost and reliability issues at once.

The system uses a Vue 3 front end and a Flask back end communicating over REST and WebSocket. On the recognition side, MTCNN handles face detection and alignment, and ResNet50 — chosen after a comparative experiment against VGG16 — extracts 512-dimensional L2-normalized feature vectors that are matched against a registered gallery using cosine similarity. Rather than forcing a binary present/absent decision, a dual-threshold design introduces a third state: results above 0.75 cosine similarity are marked present, results between 0.50 and 0.75 are flagged for manual review, and anything below 0.50 is treated as an unregistered face. This handles the practical trade-off between false acceptance and false rejection more gracefully than a single threshold would. On the LFW Top-50 subset, ResNet50 reached 86.30% validation Top-1 accuracy, 99.30% AUC-ROC, and a false acceptance rate of 1.17%, with a per-frame CPU latency of 176.5ms — all within the design targets. VGG16 was tested with a Global Average Pooling (GAP) head replacing the original three-layer FC classifier, which reduced head parameters from ~108M to ~260K and allowed batch_size=32; after full 30-epoch training it reached 74.67% validation Top-1 accuracy and 98.22% AUC-ROC, substantially better than the FC-head version. However, its 138M-parameter convolutional backbone still produces a CPU inference latency of 405ms — more than double the 200ms target — a structural constraint that no training improvement can overcome. It was therefore ruled out for deployment.

Keywords: Convolutional Neural Network; Face Recognition; Transfer Learning; Attendance System; ResNet50; VGG16

1.3 目录

第一章绪论 - 1.1 研究背景与意义 - 1.1.1 传统考勤方式的局限性 - 1.1.2 人脸识别技术的发展与应用前景 - 1.1.3 研究意义 - 1.2 国内外研究现状 - 1.2.1 人脸识别技术研究现状 - 1.2.2 智能考勤系统研究现状 - 1.2.3 现有研究的不足 - 1.3 研究内容与目标 - 1.3.1 研究内容 - 1.3.2 性能目标 - 1.4 论文结构安排

第二章相关技术综述 - 2.1 卷积神经网络基础 - 2.1.1 CNN 的基本结构 - 2.1.2 VGG16 网络结构 - 2.1.3 ResNet50 网络结构与残差连接 - 2.2 迁移学习 - 2.2.1 迁移学习原理 - 2.2.2 两

阶段微调策略 - 2.3 人脸检测：MTCNN - 2.3.1 MTCNN 基本原理 - 2.3.2 相比传统方法的优势 - 2.4 特征匹配：余弦相似度与双阈值机制 - 2.4.1 L2 归一化与余弦相似度 - 2.4.2 双阈值判断机制 - 2.5 Web 技术栈 - 2.5.1 Flask 框架 - 2.5.2 Socket.IO 实时通信 - 2.5.3 Vue 3 与 Pinia - 2.5.4 JWT 身份认证

第三章需求分析 - 3.1 系统概述 - 3.2 用户角色与权限分析 - 3.2.1 用户角色定义 - 3.2.2 权限矩阵 - 3.3 功能需求 - 3.3.1 用户管理模块（F1）- 3.3.2 学生档案模块（F2）- 3.3.3 人脸数据管理模块（F3）- 3.3.4 考勤任务模块（F4）- 3.3.5 考勤记录模块（F5）- 3.3.6 报表统计模块（F6）- 3.4 非功能需求 - 3.4.1 性能需求 - 3.4.2 安全需求 - 3.4.3 可用性需求 - 3.5 系统用例分析

第四章系统设计 - 4.1 系统总体架构 - 4.1.1 三层架构设计 - 4.1.2 技术选型说明 - 4.2 数据库设计 - 4.2.1 E-R 图 - 4.2.2 核心数据表设计 - 4.2.3 索引设计 - 4.3 AI 模块设计 - 4.3.1 模块整体架构 - 4.3.2 推理流程设计 - 4.3.3 训练模块设计 - 4.4 后端接口设计 - 4.4.1 统一响应格式 - 4.4.2 核心接口列表 - 4.4.3 JWT 权限控制实现 - 4.5 前端架构设计 - 4.5.1 路由与权限守卫 - 4.5.2 核心组件结构 - 4.5.3 实时考勤前端状态机

第五章系统实现 - 5.1 后端核心实现 - 5.1.1 项目结构与蓝图划分 - 5.1.2 应用工厂与扩展注册 - 5.1.3 JWT 权限控制实现 - 5.1.4 考勤任务 API 实现 - 5.2 WebSocket 实时识别实现 - 5.2.1 服务端帧处理逻辑 - 5.2.2 防重复出勤处理 - 5.3 AI 推理模块实现 - 5.3.1 ModelManager 单例设计 - 5.3.2 特征提取：512 维 Embedding - 5.3.3 特征库匹配与双阈值判断 - 5.4 前端核心实现 - 5.4.1 全局请求封装与错误处理 - 5.4.2 实时考勤组件实现 - 5.4.3 角色自适应菜单实现 - 5.5 本章小结

第六章系统测试与实验 - 6.1 功能测试 - 6.1.1 测试环境 - 6.1.2 功能测试用例 - 6.1.3 边界条件测试 - 6.2 AI 模型实验 - 6.2.1 实验数据集 - 6.2.2 训练配置 - 6.2.3 训练过程分析 - 6.2.4 测试集评估结果 - 6.3 VGG16 与 ResNet50 架构对比分析 - 6.3.1 架构参数对比 - 6.3.2 训练过程对比 - 6.3.3 测试集性能指标实测对比 - 6.3.4 模型选型结论 - 6.4 系统性能测试 - 6.4.1 接口响应时间 - 6.4.2 WebSocket 并发测试 - 6.5 本章小结

第七章总结与展望 - 7.1 工作总结 - 7.2 不足与局限 - 7.3 未来展望

参考文献

---

2 第一章绪论

2.1 1.1 研究背景与意义

2.1.1 1.1.1 传统考勤方式的局限性

高校课堂考勤是教学管理绕不开的一个环节。说起来很简单，但实际操作起来却挺麻烦的，尤其是班级规模不断扩大之后，这个问题就更突出了。

目前各高校用得最普遍的，一是人工点名，老师对着花名册一个一个叫，一个五六十人的大班往往要叫上十几分钟，这十几分钟其实本来可以用来讲课的，挺可惜的。二是签到表或者扫二维码，速度确实快了不少，但防作弊能力很弱——二维码发到群里转一圈，不在场的同学照样能签到，这种现象在学生中其实相当普遍。还有些学校用校园卡刷卡，硬件成本高不说，借卡代签的问题照样没解决，治标不治本。

唐琳 [15] 在研究中指出，传统考勤管理方式人力消耗大、数据误差多，需要更智能的替代方案。李国平等 [16] 也认为现有手段在效率和准确性上都还有不小的提升空间。从这几年的实际情况来看，确实到了需要认真考虑技术升级的时候了。

2.1.2 1.1.2 人脸识别技术的发展与应用前景

近年来，以深度卷积神经网络为核心的人脸识别技术取得了非常显著的突破性进展。从早期的传统方法到如今的深度学习方法，人脸识别的准确率提升是非常惊人的。DeepFace[6] 在 LFW 数据集上取得了 97.35% 的准确率，已非常接近人类识别精度（97.53%）；此后 FaceNet[5] 进一步达到 99.63%，正式超越了人类水平。人脸识别技术具有非接触、无感知、难伪造等天然优势，为解决课堂考勤难题提供了一条比较可行的技术路径。

把深度学习人脸识别技术引入高校考勤，实际价值是双重的：效率层面，几秒内完成全班识别，远快于人工点名；可信度层面，生物特征难以代签，比签到表和二维码更具说服力；而识别过程自然产生的出勤时序数据，也可以直接支持教学管理决策。更关键的是，近年来摄像头和 CPU 服务器的成本持续下降，这意味着不需要专用 GPU 硬件就能把这套系统跑起来，落地门槛比之前低了很多。

2.1.3 1.1.3 研究意义

本研究的意义，个人觉得可以从几个不同角度来理解。

从实际应用来说，做这个系统的直接目的是减轻老师的考勤负担，同时让出勤数据更可信一些，这对教务管理是有实际价值的。从工程实践来说，把 WebSocket 实时通信和 AI 推理模块集成到一套完整的 Web 系统里，本身就有一定的挑战性，验证这套路子是否跑得通，对类似项目有一定的参考意义。从实验研究来说，在小样本人脸识别场景下，VGG16 和 ResNet50 这两种架构的表现差异其实挺大的，这个对比结果对于理解不同模型在资源受限环境下的适用性，应该是有些参考价值的。

2.2 1.2 国内外研究现状

2.2.1 1.2.1 人脸识别技术研究现状

人脸识别技术走到今天，经历了一个从手工特征到深度学习的比较大的转变。

早期的方法以 Eigenfaces[11] 为代表，Turk 和 Pentland 在 1991 年提出用 PCA 把人脸图像投影到特征脸空间来做识别，在当时是很有创意的思路。后来又出现了基于 LDA 的 Fisherfaces 和 LBP 等方法，在光照均匀、姿态变化不大的受控环境下效果还可以，但一旦条件稍复杂一些，识别率就掉得很厉害——鲁棒性不足是这类方法绕不开的短板。

深度学习的出现改变了这个局面。2014 年 Taigman 等人的 DeepFace[6] 把 CNN 引入人脸识别，在 LFW 上直接做到了 97.35%，跟人类精度（97.53%）差得极少，这在当时相当轰动。同年 Simonyan 等提出的 VGGNet[2] 虽然主要是通用分类模型，但其特征提取能力非常强，后来被大量借用于人脸任务。2015 年是个丰收年，何凯明等的 ResNet[1] 用残差连接解决了深层网络难以训练的问题，而 Schroff 等的 FaceNet[5] 则把三元组损失用得很漂亮，学到了高度紧凑的特征空间表示，在多个评测集上都拿了最好成绩，同期 Ioffe 和 Szegedy 的批归一化 [9] 也开始流行，让训练稳定了很多。到了 2019 年，Deng 等提出的 ArcFace[4] 用加性角度间隔损失把类间分离度进一步提升，成为目前工业界最常用的训练方案之一。

余璀璨和李慧斌 [12] 的综述对这一发展脉络做了系统梳理，也指出迁移学习在数据有限的场景下价值突出。Song 等 [14] 的工作也从侧面印证了 CNN 在人脸相关任务中的特征表达

能力。

2.2.2 1.2.2 智能考勤系统研究现状

国内研究：国内在智能考勤领域的研究已经比较丰富了。唐琳 [15] 设计并实现了基于人脸识别技术的课堂考勤管理系统，采用 Flask 后端和前端相结合的架构，系统功能比较完整。李国平和李小纳 [16] 研究了基于人脸识别的智慧教室学生考勤系统，重点关注了嵌入式设备上的部署问题。此外，屈东东等 [13] 在轻量化人脸识别算法方面的研究表明，改进 MobileNet 等轻量级网络可以在嵌入式平台上实现实时识别，为资源受限场景提供了新的思路。

国外研究：国外对于智能考勤系统的研究也有不少，但是从目前可查阅的文献来看，很多研究主要聚焦在识别算法本身的精度提升上，对于完整的 Web 系统工程架构的讨论相对较少，尤其是涉及角色权限管理和实时 WebSocket 推送的端到端系统实现的研究就更少见了。

2.2.3 1.2.3 现有研究的不足

看了这些研究之后，感觉现有工作还是有几个地方不太令人满意。很多系统只关注识别算法本身，系统功能相对单薄，角色权限管理、教务业务闭环这些工程层面的内容很少有人认真做。把 WebSocket 实时推送和 AI 推理真正深度集成的工作也不多见，大多停留在离线识别或者简单的接口调用层面。另外，针对课堂这种小样本场景，VGG16 和 ResNet50 等主流架构之间的系统性比较实验也比较稀缺，缺少能直接参考的数据。

本文主要就是针对这几个方向做了一些补充，试图做出一套功能相对完整、实时性有保证的考勤系统，同时通过实验给出两种架构在这类场景下的对比数据。

2.3 1.3 研究内容与目标

2.3.1 1.3.1 研究内容

本文的工作可以拆成四块来看。

第一块是 AI 识别模块，也是整个系统的技术核心：研究 VGG16 与 ResNet50 在人脸识别任务上的迁移学习策略，设计并验证了 “冻结主干 → 全局微调” 的两阶段训练方案，实现特征提取与余弦相似度匹配推理链路。

第二块是后端服务，围绕 Flask 框架展开：设计 RESTful API，集成 Flask-SocketIO 做实时推送，用 JWT 实现管理员、教师、辅导员、学生四种角色各自隔离的权限控制。

第三块是前端界面，基于 Vue 3 + Element Plus：实现包含学生管理、人脸采集、实时考勤监控、报表统计在内的响应式 Web 管理界面，覆盖完整业务流程。

第四块是系统集成与实验验证：在 LFW 数据集 Top-50 子集上跑 VGG16 与 ResNet50 的对比实验，评估 Top-1 准确率、AUC-ROC、FAR 等多项指标，验证系统端到端的实际性能。

2.3.2 1.3.2 性能目标

根据任务书要求，本系统需要达到如下量化指标：

指标	目标值
Top-1 识别准确率	$\geqslant 85\%$
Precision / Recall / F1	$\geqslant 0.85$
AUC-ROC	$\geqslant 0.90$
误识率（FAR）	$\leqslant 5\%$
拒识率（FRR）	$\leqslant 10\%$
单帧推理延迟（CPU）	$\leqslant 200$ms

2.4 1.4 论文结构安排

本文共分为七章，各章内容安排如下：

第一章是绪论，交代研究背景、国内外现状、研究内容与目标。

第二章是相关技术综述，把这篇论文用到的技术——CNN 架构、迁移学习、MTCNN、余弦相似度，以及 Web 技术栈——统一做一个原理层面的梳理，方便后续章节直接引用而不需要每次从头解释。

第三章是需求分析，从功能需求、非功能需求和用例三个角度把系统要做什么说清楚。

第四章是系统设计，覆盖总体架构、数据库表结构、AI 模块设计和接口定义。

第五章是系统实现，讲的是各模块的关键代码怎么写的、有哪些值得注意的细节。

第六章是系统测试与实验，包括功能黑盒测试结果和 AI 模型对比实验的完整数据分析。

第七章是总结与展望，把做了什么、没做好什么、以后可以怎么改都交代一遍。

---

3 第二章相关技术综述

3.1 2.1 卷积神经网络基础

3.1.1 2.1.1 CNN 的基本结构

卷积神经网络（Convolutional Neural Network, CNN）是计算机视觉领域用得最多的深度学习模型。1998 年 LeCun 等 [10] 提出的 LeNet 是最早的代表性工作，虽然那时候的网络规模和现在没法比，但基本的结构思路是一脉相承的。

CNN 最核心的部件是卷积层。它的工作方式是用一个小的卷积核在输入特征图上滑动，每次只看一个局部区域——这种 “局部连接 + 权值共享” 的设计，使得参数量远远少于全连接的方式。数学上，卷积操作可以写成：

$$Y_{i,j} = \sum_m \sum_n X_{i+m,j+n} \cdot W_{m,n} + b$$

其中 $W_{m,n}$ 是卷积核权重，b 是偏置项。

卷积之后通常跟着池化层做降采样，压缩特征图尺寸的同时保留主要信息。最大池化用得比平均池化多，因为它更善于保留显著特征。网络末端的全连接层负责把前面提取到的空间特征整合起来输出分类结果——在人脸识别场景里，这个部分通常会改造成输出固定维度向量的特征嵌入层，本文用的是 512 维。

值得一提的是批归一化技术 [9]，Ioffe 和 Szegedy 在 2015 年提出它，通过对每个 mini-batch 的中间输出做归一化来缓解内部协变量偏移，训练稳定性大幅提升，现在几乎成了标配。

3.1.2 2.1.2 VGG16 网络结构

VGGNet[2] 是由牛津大学视觉几何组提出的，以其简洁一致的 3×3 卷积设计著称。VGG16 包含 13 个卷积层和 3 个全连接层，参数总量约为 1.38 亿。VGG16 的核心设计思路是用多个 3×3 卷积堆叠来等效更大感受野（两个 3×3 等效一个 5×5），在减少参数的同时引入更多非线性，特征提取能力较强。但其参数量庞大（约 1.38 亿）、计算成本较高，在资源受限的部署场景下存在明显的内存和速度瓶颈。

本文在 VGG16 基础上，将原始的 7×7 自适应平均池化替换为全局平均池化（GAP），并将三层全连接分类头（~1.08 亿参数）替换为更轻量的结构：

$$\text{GAP}(512\times7\times7 \to 512) \to \text{FC}(512 \to 512) \to \text{ReLU} \to \text{Dropout} \to \text{FC}(512 \to C)$$

GAP 将最后一个卷积块的输出从 25088 维压缩至 512 维，分类头参数量由原来的约 1.08 亿骤降至约 26 万，内存占用大幅降低，使得 batch_size 可以从受内存限制的 4 提升至 32。其中倒数第二层的 512 维输出作为人脸特征向量使用。

3.1.3 2.1.3 ResNet50 网络结构与残差连接

ResNet[1] 由何凯明等于 2015 年提出，通过引入残差学习（Residual Learning）解决了深层网络训练中的退化问题。其核心创新是跳跃连接（Skip Connection），使得网络只需要学习残差映射而不是完整映射：

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}$$

其中 $\mathcal{F}(\mathbf{x})$ 是需要学习的残差部分，$\mathbf{x}$ 是直接传递的恒等映射。这样的设计极大地缓解了梯度消失问题，使得训练非常深的网络成为可能。

ResNet50 采用 Bottleneck 结构（1×1→3×3→1×1 卷积堆叠），网络共 50 层，参数量约为 2500 万，远少于 VGG16 的 1.38 亿，在精度和效率上取得了更好的平衡。本文将 ResNet50 的全连接头替换为：

$$\text{FC}(2048 \to 512) \to \text{ReLU} \to \text{Dropout} \to \text{FC}(512 \to C)$$

与 VGG16（GAP 改造后分类头约 26 万参数）相比，ResNet50 的分类头参数量约 107 万，两者在分类头上差距已不大；但 ResNet50 的卷积主干（2500 万参数）远轻于 VGG16（约 1.38 亿参数），这一结构性差异直接决定了推理延迟和训练内存的不同。

3.2 2.2 迁移学习

3.2.1 2.2.1 迁移学习原理

迁移学习（Transfer Learning）是将在源域（Source Domain）上学习到的知识迁移到目标域（Target Domain）的机器学习方法 [8]。Pan 和 Yang 的综述性工作 [8] 对迁移学习进行了系统性的分类和梳理，将其分为归纳式迁移、直推式迁移和无监督迁移三类。在计算机视觉领域，基于 ImageNet 预训练模型的微调是最常用的迁移学习方式。

在人脸识别领域，用 ImageNet 预训练模型来迁移是非常合理的。毕竟 ImageNet 上已经学到了大量纹理、边缘、形状等底层特征，这些东西对人脸任务同样有用，完全可以直接继承过来而不必从零学起。更实际的好处是，这种方式对训练数据量的要求低得多——本系统每类学生的人脸图片只有几十张，从头训练基本不可能收敛，迁移学习在这里几乎是唯一可行的路线。另外收敛速度也快很多，在本文的实验里，冻结主干只训练分类头的第一阶段，仅用 10 轮就能达到 45% 左右的验证准确率，如果从头训练同样的数据量大概率连 10% 都很难到。

3.2.2 2.2.2 两阶段微调策略

本系统采用两阶段迁移学习策略，这是根据实际训练环境和数据特点设计的：

阶段一：冻结主干，训练分类头（Epochs 1~10）

在这一阶段，冻结预训练骨干网络的所有卷积层参数，只训练新增的分类头。使用较高的学习率（$lr = 10^{-3}$），让随机初始化的分类头能够在 ImageNet 特征空间上快速收敛，避免太大的梯度破坏主干网络已经学到的特征表示，这个策略也是文献中比较常见的做法。

阶段二：解冻全部，端到端微调（Epochs 11~30）

在第二阶段，解冻所有参数，切换为 SGD+Momentum 优化器（$lr = 10^{-4}$），并配合余弦退火学习率调度，在保留预训练特征的基础上对整个网络进行细粒度调整，使特征提取器能够更好地适配人脸识别这一目标任务。

在微调阶段选择 SGD 而不是 Adam，主要有两方面考虑：一是 SGD 在 CPU 环境下的内存占用大约是 Adam 的三分之一，对内存紧张的部署环境更友好；二是有研究表明，在微调任务上 SGD+Momentum 相比 Adam 往往具有更好的泛化性能，余璀璨和李慧斌 [12] 也在综述中提到了类似的观察。

3.3 2.3 人脸检测：MTCNN

3.3.1 2.3.1 MTCNN 基本原理

MTCNN（Multi-task Cascaded Convolutional Networks）[3] 是 Zhang 等于 2016 年提出的一种基于级联 CNN 的人脸检测与关键点定位方法，由三个子网络串联构成：

P-Net（Proposal Network）：快速扫描全图，在多个尺度上生成候选人脸框（Bounding Box）和面部关键点的初始估计，同时过滤掉大量背景区域，计算量比较小；

R-Net（Refine Network）：接收 P-Net 输出的候选框，进行更精细的筛选和回归，进一步提高检测框的定位精度；

O-Net（Output Network）：最终输出精确的人脸边界框和 5 个面部关键点（双眼中心、鼻尖、双嘴角）坐标。

三个网络级联使用，形成由粗到精的多阶段检测流程，在保证精度的同时维持了较快的检测速度。Zhang 等 [3] 的原文实验表明，MTCNN 在 FDDB 和 WIDER FACE 等标准人脸检测数据集上均取得了优于当时同类方法的检测精度，且单张图片的平均检测耗时不超过 16ms（VGA 分辨率），满足实时应用需求。

3.3.2 2.3.2 相比传统方法的优势

传统的 Haar Cascade 人脸检测方法对光照变化和角度变化比较敏感，误检率较高，这个是业界比较公认的问题了。MTCNN 通过多任务联合学习（同时学习分类、框回归和关键点定位三个任务），在多尺度、多角度、复杂光照条件下均表现出比较强的鲁棒性。本系统选用 MTCNN 作为人脸检测模块，能够较好地应对课堂摄像头采集图像中光照不均、多角度等实际挑战。

3.4 2.4 特征匹配：余弦相似度与双阈值机制

3.4.1 2.4.1 L2 归一化与余弦相似度

提取的 512 维特征向量经 L2 归一化后，欧氏距离与余弦相似度在数值上是等价的。对于归一化向量 $\hat{f}_i = f_i / \|f_i\|_2$，两向量之间的余弦相似度为：

$$\mathrm{sim}(\hat{f}_q, \hat{f}_k) = \hat{f}_q \cdot \hat{f}_k = \sum_{d=1}^{512} \hat{f}_q^{(d)} \cdot \hat{f}_k^{(d)}$$

取值范围在 $[-1, 1]$ 之间，值越大表示越相似。ArcFace[4] 等方法也是将特征投影到超球面上利用角度度量来计算相似度，本文的 L2 归一化余弦相似度与此思路一致。归一化后的批量匹配可以用矩阵点积高效计算：$\mathbf{S} = \hat{F}_{\text{query}} \cdot \hat{F}_{\text{db}}^T$，在百人规模下匹配耗时不足 1ms，计算效率很高。

3.4.2 2.4.2 双阈值判断机制

单一阈值在实际应用中存在一个问题，就是拒识率和误识率很难同时优化得很好，总是要在二者之间做取舍。本系统设计了一个双阈值机制来缓解这一矛盾：

相似度 s	判断结果	含义
$s \geq 0.75$	出勤（present）	高置信度识别成功
$0.50 \leq s < 0.75$	待核实（unverified）	识别结果存疑，待人工复核

相似度 s	判断结果	含义
$s < 0.50$	未知（rejected）	非注册学生，忽略

“待核实” 这个中间状态引入了人工监督的环节，在保证系统自动化的同时降低了误识风险，更贴近实际教学管理的需求。从实验结果来看，这个机制对于高拒识率的情况有比较明显的弥补效果。

3.5 2.5 Web 技术栈

3.5.1 2.5.1 Flask 框架

Flask 是基于 Python 的轻量级 WSGI Web 框架，以灵活的蓝图（Blueprint）机制和丰富的扩展生态著称。本系统使用了 Flask-JWT-Extended、Flask-SQLAlchemy 和 Flask-SocketIO 这几个主要扩展，分别处理认证、数据库操作和实时通信。Flask 的轻量特性适合快速构建 RESTful API 服务，而且 Python 生态与 PyTorch 的 AI 推理模块可以无缝集成，这也是选择 Flask 而不是其他框架的一个重要原因。

3.5.2 2.5.2 Socket.IO 实时通信

传统 HTTP 轮询在实时场景下有延迟高、服务器压力大等不足，不太适合频繁更新的场景。Socket.IO 基于 WebSocket 协议实现全双工实时通信，支持房间（Room）机制，可以将识别结果精确推送到指定考勤任务的监控界面，有效降低了端到端延迟。每个考勤任务对应一个独立的 Socket.IO 房间，不同任务之间的识别结果互不干扰。

3.5.3 2.5.3 Vue 3 与 Pinia

Vue 3 基于 Composition API 提供了更灵活的代码组织方式，配合 Pinia 状态管理库和 Element Plus UI 组件库，可以实现响应式的单页面应用（SPA）。Pinia 的 `attendanceStore` 集中管理 WebSocket 连接、摄像头帧抓取和识别状态，将 UI 层与业务逻辑有效解耦，使代码结构更加清晰。

3.5.4 2.5.4 JWT 身份认证

JSON Web Token（JWT）是一种无状态的身份认证方案，服务端不需要存储 Session 信息，Token 由 Header、Payload、Signature 三部分组成，其中 Payload 中携带用户 ID 和角色信息，便于实现无状态的分布式权限控制。本系统 Token 有效期为 24 小时，前端解析 exp 字段实现本地过期检测，过期时自动跳转到登录页面。

---

4 第三章需求分析

4.1 3.1 系统概述

本系统面向高校课堂场景，旨在通过人脸识别技术替代传统手工考勤方式，实现自动化、数字化的出勤管理。系统需要支持实时摄像头识别、多角色权限管理和完整的考勤数据生命周期管理，同时具备足够的鲁棒性来应对课堂摄像头采集图像中常见的光照不均、多角度、遮挡等问题。

4.2 3.2 用户角色与权限分析

4.2.1 3.2.1 用户角色定义

系统定义了四类用户角色，每类角色有不同的职责范围：

角色	角色标识	职责描述
系统管理员	admin	负责用户账号管理、学生档案维护、人脸数据采集、模型管理
教师	teacher	发起考勤任务、监控实时识别、查询本课程考勤记录
辅导员	counselor	查询全院学生考勤数据、生成报表、关注缺勤预警
学生	student	查询本人考勤记录，了解出勤状态

4.2.2 3.2.2 权限矩阵

功能模块	管理员	教师	辅导员	学生
用户账号管理	√	—	—	—
学生信息录入/修改	√	—	—	—
学生列表查询	√	√	√	—
人脸数据采集与管理	√	—	—	—
模型训练触发	√	—	—	—
发起/结束考勤任务	√	√	—	—
考勤任务列表	√	√	√	—
考勤记录纠错	√	√	—	—
本人考勤查询	—	—	—	√
报表统计	√	√	√	—
数据概览 Dashboard	√	√	√	√

4.3 3.3 功能需求

4.3.1 3.3.1 用户管理模块（F1）

- **F1.1** 用户登录：支持用户名和密码登录，登录成功后签发 JWT Token，有效期 24 小时；
- **F1.2** 用户 CRUD：管理员可以创建、查询、修改、禁用用户账号，但不可删除当前登录的账号；
- **F1.3** 权限控制：所有 API 接口基于 JWT Payload 中的 role 字段进行权限校验，权限不足时统一返回 403 状态码。

4.3.2 3.3.2 学生档案模块（F2）

- **F2.1** 分页查询：支持按姓名（模糊匹配）、班级、是否注册人脸进行筛选；
- **F2.2** 单条录入：管理员填写学号、姓名、班级、院系等基本信息；

- **F2.3** Excel 批量导入：支持上传标准格式的 Excel 文件，系统自动跳过重复学号，并返回导入结果的统计信息；
- **F2.4** 选课管理：支持为学生批量绑定课程，这个信息用于确定每次考勤任务的学生范围。

4.3.3 3.3.3 人脸数据管理模块（F3）

- **F3.1** 人脸图片采集：通过摄像头实时截帧或者上传图片，调用 MTCNN 检测后裁剪存储；
- **F3.2** 人脸图片管理：查看和删除已录入的人脸图片，展示每个学生的图片数量；
- **F3.3** 模型训练触发：管理员一键触发重新训练，后台异步执行，支持查询训练进度状态；
- **F3.4** 特征库更新：训练完成后自动提取所有已注册学生的平均特征向量，更新特征库文件，供后续推理使用。

4.3.4 3.3.4 考勤任务模块（F4）——核心功能

这个模块是整个系统最核心的部分，需要重点保证功能正确性和实时性。

- **F4.1** 创建考勤任务：教师选择课程和考勤日期创建任务，初始状态为 “待开始”；
- **F4.2** 实时考勤识别：进入考勤界面后开启摄像头，前端以 1fps 频率抓取视频帧，通过 WebSocket 发送到服务端；服务端调用 MTCNN 检测人脸，经 ResNet50 提取特征后进行余弦相似度匹配，再将识别结果实时推送回前端；
- **F4.3** 出勤统计：实时更新出勤人数、总人数、出勤率等统计数据；
- **F4.4** 结束考勤：停止任务后，自动为未被识别到的学生生成 “缺勤” 记录；
- **F4.5** 任务管理：查询考勤任务列表，允许删除处于 “待开始” 状态的任务。

4.3.5 3.3.5 考勤记录模块（F5）

- **F5.1** 记录查询：支持按任务、课程、日期范围、状态进行筛选，其中学生角色只能查询本人的考勤记录；
- **F5.2** 状态纠错：教师可以将 “待核实” 状态的记录手动修改为 “出勤” 或 “缺勤”；

- **F5.3** 学生自查：学生可以查看本人的出勤率统计，包括出勤、缺勤、待核实各状态的占比。

4.3.6 3.3.6 报表统计模块（F6）

- **F6.1** 数据概览 Dashboard：展示学生总数、今日考勤场次、今日出勤率、待核实记录数、近 7 日出勤趋势折线图以及缺勤预警榜；
- **F6.2** 课程出勤统计：按课程和日期范围查询出勤率分布，展示平均、最高、最低出勤率等统计卡片；
- **F6.3** 缺勤预警：列出缺勤次数超过预设阈值的学生名单，方便辅导员及时跟进。

4.4 3.4 非功能需求

4.4.1 3.4.1 性能需求

指标	要求
识别响应时间	单帧端到端延迟 ⩽ 200ms（含检测、特征提取和匹配全流程）
并发考勤任务	支持同时运行多个考勤任务，房间隔离互不干扰
页面加载时间	首屏加载 ⩽ 3 秒（开发环境）

4.4.2 3.4.2 安全需求

- 密码采用 bcrypt 加盐哈希存储，禁止以明文形式存储在数据库中；
- JWT Token 设置 24 小时过期时间，前端解析 exp 字段实现本地过期检测；
- 所有 API 接口都要进行角色鉴权，防止越权访问；
- 人脸图片文件存储于服务器端，前端通过认证接口访问，不直接暴露文件存储路径。

4.4.3 3.4.3 可用性需求

- 界面操作应该符合直觉，对于删除、结束考勤等不可逆操作需要设置二次确认弹窗；

- 权限不足时给出明确的文字提示，同时不展示用户无权限操作的功能入口；
- 摄像头不可用时提供清晰的错误提示信息，并支持手动上传图片作为降级方案。

4.5 3.5 系统用例分析

系统包含多个用例场景，其中最核心的三个用例是：**实时考勤识别**（教师发起 → 前端采帧 →WebSocket 推理 → 结果展示）、**人脸数据采集**（管理员录入 →MTCNN 检测 → 裁剪存储 → 等待训练）和**学生查询考勤**（学生登录 → 查询本人记录 → 查看出勤率统计）。核心的实时考勤流程时序如下所示：

```
教师            前端              后端API        WebSocket        AI推理模块

  点击"开始"
      →
                POST /tasks/:id/start
                      →
                                  状态→running
              ←
                开启摄像头，1fps抓帧
                emit("frame", base64)
                                →
                                  收到帧
                                              →
                                                MTCNN检测
                                                ResNet50提取
                                                余弦匹配
                                ←
                                  写入AttendanceRecord
                                  emit("recognized", result)
              ←
                更新出勤列表和统计
 ←
```

5 第四章系统设计

5.1 4.1 系统总体架构

5.1.1 4.1.1 三层架构设计

系统采用前后端分离加 AI 推理模块的三层架构:

```
                 前端层 (Vue 3 SPA)
Element Plus UI  Pinia Store  socket.io-client
用户管理 / 学生管理 / 考勤监控 / 报表统计

                  HTTP/REST        WebSocket

                 后端层 (Flask 3.x)
JWT鉴权  Blueprint路由  SQLAlchemy ORM
/api/auth  /api/students  /api/attendance
/api/faces  /api/reports  /api/users

                   函数调用

      AI推理模块                      MySQL 8.0
   MTCNN检测  ResNet50提取            数据持久层
   特征库匹配  双阈值判断
```

前端层负责用户交互与界面渲染，通过 Vite 开发服务器代理将`/api`请求转发到后端；后端层处理业务逻辑、数据库操作和权限验证，通过 Flask-SocketIO 维护 WebSocket 长连接；AI 推理模块以 Python 类库形式嵌入在后端进程中，通过 ModelManager 单例提供人脸检测、特征提取和相似度匹配服务，避免了跨进程调用的额外开销。

5.1.2 4.1.2 技术选型说明

层次	技术选型	选型理由
前端框架	Vue 3 + Vite 5	Composition API 提升代码组织性，Vite 热更新速度快
UI 组件库	Element Plus	企业级组件齐全，与 Vue 3 深度集成
状态管理	Pinia	轻量，比 Vuex 更简洁
后端框架	Flask 3.x	轻量灵活，生态丰富，与 PyTorch 无缝集成
数据库	MySQL 8.0	成熟稳定，支持事务，适合结构化教务数据
人脸检测	MTCNN（facenet-pytorch）	多尺度精确检测，支持关键点定位
特征提取	ResNet50 / VGG16	主流迁移学习骨干，有对比实验价值
实时通信	Flask-SocketIO + eventlet	支持 WebSocket 房间机制，易与 Flask 集成
认证方式	Flask-JWT-Extended	无状态 Token，适合前后端分离架构

5.2 4.2 数据库设计

5.2.1 4.2.1 E-R 图

系统包含 7 张核心数据表，主要实体关系如下：

- `User` 1:N → `AttendanceTask`（教师发起考勤任务）
- `Student` M:N ↔ `Course`（通过 `StudentCourse` 中间表实现选课关联）
- `Course` 1:N → `AttendanceTask`（一门课程对应多次考勤任务）
- `AttendanceTask` 1:N → `AttendanceRecord`（每次任务产生多条记录）
- `Student` 1:N → `AttendanceRecord`（一个学生对应多条考勤记录）
- `Student` 1:N → `FaceImage`（一个学生对应多张人脸图片）

5.2.2 4.2.2 核心数据表设计

users 用户表

字段	类型	约束	说明
id	INT	PK, AUTO_INCREMENT	用户 ID
username	VARCHAR(50)	UNIQUE, NOT NULL	登录账号
password_hash	VARCHAR(128)	NOT NULL	bcrypt 散列值
role	ENUM	NOT NULL	admin/teacher/counselor/studen
real_name	VARCHAR(50)	NOT NULL	真实姓名
is_active	TINYINT(1)	DEFAULT 1	账号启用状态

attendance_tasks 考勤任务表

字段	类型	约束	说明
id	INT	PK	任务 ID
course_id	INT	FK→courses.id	关联课程
teacher_id	INT	FK→users.id	发起教师
task_date	DATE	NOT NULL	考勤日期
status	ENUM	NOT NULL	pending/running/finished
total_students	INT	DEFAULT 0	课程总人数（快照）
present_count	INT	DEFAULT 0	已出勤人数（实时更新）
start_time	DATETIME		开始时间
end_time	DATETIME		结束时间

attendance_records 考勤记录表

字段	类型	约束	说明
id	INT	PK	记录 ID
task_id	INT	FK→attendance_tasks.id	关联任务
student_id	INT	FK→students.id	关联学生
status	ENUM	NOT NULL	present/absent/unverified
confidence	FLOAT		识别置信度（余弦相似度值）
recognized_at	DATETIME		识别时间戳
verified_by	INT	FK→users.id, NULL	人工复核操作者
note	VARCHAR(255)		备注

5.2.3 4.2.3 索引设计

针对高频查询场景，设计了如下关键索引：

表	索引字段	类型	优化场景
users	username	UNIQUE	登录时账号查询
students	student_id	UNIQUE	识别结果写库时快速定位
students	class_name	INDEX	按班级查询学生列表
attendance_tasks	(course_id, task_date)	INDEX	按课程和日期查询任务
attendance_tasks	status	INDEX	按状态筛选任务
attendance_records	(task_id, student_id)	INDEX	防重复写入检查
face_images	student_id	INDEX	查询学生人脸图片

5.3 4.3 AI 模块设计

5.3.1 4.3.1 模块整体架构

AI 模块由五个子模块组成，通过 `ModelManager` 单例统一对外提供服务：

子模块	职责
`FaceDetector`	封装 MTCNN，输入帧图像，输出人脸 Bounding Box 列表
`FacePreprocessor`	按框裁剪人脸区域，缩放至 224×224，ImageNet 归一化
`FaceRecognizer`	加载训练好的 VGG16/ResNet50 权重，前向传播到 512 维特征层
`FeatureDatabase`	加载特征库 numpy 矩阵，执行 L2 归一化和批量余弦相似度匹配
`ModelManager`	单例，协调上述四个模块，提供 `recognize_faces(frame)` 接口

5.3.2 4.3.2 推理流程设计

完整的推理流程如下：

```
输入：BGR图像帧（OpenCV格式）
```

```
    FaceDetector.detect()
[MTCNN检测] → 返回[(x1,y1,x2,y2), ...]

    FacePreprocessor.process()
[裁剪+缩放+归一化] → 返回 Tensor [N, 3, 224, 224]

    FaceRecognizer.extract()
[ResNet50前向传播, 截断到fc[:2]] → 512维向量 [N, 512]

    L2归一化
[向量 / ||向量|| ] → 单位向量 [N, 512]

    FeatureDatabase.match()
[矩阵点积] S = Query × DB^T → 相似度矩阵 [N, M]

    双阈值判断
[s 0.75 → present; 0.5 s<0.75 → unverified; s<0.5 → reject]


输出: [{student_id, student_db_id, status, confidence}, ...]
```

5.3.3 4.3.3 训练模块设计

训练脚本 `model/train.py` 实现两阶段迁移学习:

数据集组织: 使用 `torchvision.datasets.ImageFolder` 读取 `dataset/faces/train/`目录，文件夹名即类别标签，不需要额外的标注文件；验证集从 `dataset/faces/val/`读取。

数据增强（仅用于训练集）: 先将图像 Resize 至 256×256，再随机裁剪至 224×224、随机水平翻转、颜色抖动（亮度/对比度/饱和度各 ±30%）、随机旋转（±15°）、ImageNet 均值标准差归一化。测试和验证时直接 Resize 至 224×224 后归一化，不做随机增强操作。

两阶段策略: Phase 1（前 10 轮）冻结骨干，Adam 优化器 $lr = 10^{-3}$；Phase 2（后 20 轮）解冻全部，SGD+Momentum 优化器 $lr = 10^{-4}$，余弦退火调度。

最优模型保存：每当验证集准确率创历史最高时保存权重文件，同时把 num_classes、best_acc、best_epoch 记录到 JSON 元信息文件，这样推理时可以自动加载对应的模型信息。

5.4 4.4 后端接口设计

5.4.1 4.4.1 统一响应格式

所有接口统一返回如下 JSON 结构，方便前端统一处理：

```
{
  "code": 200,
  "message": "success",
  "data": { ... },
  "request_id": "a1b2c3d4"
}
```

分页接口在 `data` 中额外包含 `pagination` 对象：`{ "total": 100, "page": 1, "per_page": 20, "pages": 5 }`

5.4.2 4.4.2 核心接口列表

认证模块 `/api/auth`

方法	路径	权限	说明
POST	/login	公开	用户名密码登录，返回 JWT Token
POST	/change-password	登录用户	修改密码

考勤模块 `/api/attendance`（核心）

方法	路径	权限	说明
POST	/tasks	admin/teacher	创建考勤任务

方法	路径	权限	说明
GET	/tasks	admin/teacher/counselor	查询任务列表（支持过滤）
POST	/tasks/:id/start	admin/teacher	开始考勤，记录 total_students 快照
POST	/tasks/:id/stop	admin/teacher	结束考勤，批量生成 absent 记录
DELETE	/tasks/:id	admin/teacher	删除待开始任务
GET	/records	登录用户	查询考勤记录（学生只查本人）
PUT	/records/:id	admin/teacher	手动修正考勤状态

WebSocket 事件（Socket.IO）

事件名	方向	说明
`join_task`	客户端 → 服务端	加入考勤任务房间
`frame`	客户端 → 服务端	发送 base64 编码视频帧
`recognized`	服务端 → 客户端	推送识别结果
`task_update`	服务端 → 客户端	推送任务统计数据

5.4.3 4.4.3 JWT 权限控制实现

```python
# 核心装饰器设计
def require_roles(*roles):
    def decorator(f):
        @wraps(f)
        def wrapper(*args, **kwargs):
            verify_jwt_in_request()
            claims = get_jwt()
            if claims.get('role') not in roles:
                return error_response('权限不足', 403)
            return f(*args, **kwargs)
        return wrapper
    return decorator
```

5.5 4.5 前端架构设计

5.5.1 4.5.1 路由与权限守卫

前端路由基于 Vue Router，每个路由配置 `meta.roles` 字段来声明允许访问的角色。路由守卫在每次导航前进行三重检查：第一检查 JWT 本地是否过期（解析 exp 字段）；第二检查登录状态；第三检查当前用户角色是否在目标路由的 meta.roles 中。权限不足时重定向到`/dashboard` 而不是显示错误页，这样用户体验会好一些。

5.5.2 4.5.2 核心组件结构

```
src/
  layouts/DefaultLayout.vue     # 侧边栏导航 + 顶部用户信息（角色自适应菜单）
  stores/
      auth.js                  # JWT Token管理、用户信息、登出
      attendance.js            # Socket.IO连接、摄像头帧发送、识别状态
  api/
      request.js               # axios封装，统一401/403处理
      attendance.js            # 考勤相关接口
      ...                      # 其他模块接口
  views/
      attendance/
          AttendanceList.vue   # 考勤任务列表（角色自适应操作按钮）
          AttendanceTask.vue   # 实时考勤监控（摄像头+识别结果）
          MyAttendance.vue     # 学生：个人考勤记录查询
      ...
```

5.5.3 4.5.3 实时考勤前端状态机

考勤任务在前端维护三种状态，通过 `attendanceStore` 进行管理：

```
pending（待开始）
      点击"开始" → POST /tasks/:id/start
```

```
running（识别中）
      开启摄像头定时器（1fps）
      emit("frame", base64)
      on("recognized") → 更新识别列表
      on("task_update") → 更新统计数据
      点击"结束" → POST /tasks/:id/stop

finished（已结束）
      关闭摄像头、断开WebSocket

（返回列表）
```

---

6 第五章系统实现

6.1 5.1 后端核心实现

6.1.1 5.1.1 项目结构与蓝图划分

后端采用 Flask Blueprint 机制将各业务模块解耦，使得不同功能模块可以独立开发和维护。项目核心目录结构如下：

```
backend/
  run.py                          # 应用入口，初始化Flask与SocketIO
  app/
      extensions.py               # db、jwt、socketio等扩展单例
      models/                     # SQLAlchemy ORM模型
          user.py                 # User, UserRole枚举
          student.py              # Student, StudentCourse
          attendance.py           # AttendanceTask, AttendanceRecord
          face_image.py           # FaceImage
      api/                        # RESTful蓝图
          auth.py                 # /api/auth: 登录、修改密码
```

```
        users.py                # /api/users: 用户管理
        students.py             # /api/students: 学生档案
        faces.py                # /api/faces: 人脸数据、模型训练
        attendance.py           # /api/attendance: 任务与记录
        reports.py              # /api/reports: 报表统计
    services/                   # 业务逻辑层（与API层解耦）
    ai/                         # AI推理模块
        face_recognizer.py      # FaceDetector + Embedder
        model_manager.py        # ModelManager单例
    ws/
        handlers.py             # WebSocket事件处理
model/
    train.py                    # 训练脚本（两阶段迁移学习）
    evaluate.py                 # 测试集评估脚本
```

6.1.2 5.1.2 应用工厂与扩展注册

采用工厂函数模式创建 Flask 实例，这个设计可以避免循环导入的问题，同时也方便支持多环境配置：

```python
# run.py（简化版）
from flask import Flask
from flask_socketio import SocketIO
from app.extensions import db, jwt, socketio

def create_app():
    app = Flask(__name__)
    app.config.from_object('config.DevelopmentConfig')

    db.init_app(app)
    jwt.init_app(app)
    socketio.init_app(app, cors_allowed_origins="*",
                      async_mode='eventlet', logger=False)

    from app.api.auth import auth_bp
```

```
    from app.api.attendance import attendance_bp
    from app.api.faces import faces_bp
    from app.api.reports import reports_bp
    app.register_blueprint(auth_bp,       url_prefix='/api/auth')
    app.register_blueprint(attendance_bp, url_prefix='/api/attendance')
    app.register_blueprint(faces_bp,      url_prefix='/api/faces')
    app.register_blueprint(reports_bp,    url_prefix='/api/reports')
    return app

if __name__ == '__main__':
    app = create_app()
    socketio.run(app, host='0.0.0.0', port=5000, debug=True)
```

6.1.3 5.1.3 JWT 权限控制实现

系统使用 Flask-JWT-Extended 实现无状态 Token 认证。登录成功后，Token Payload 中携带 `sub`（用户 ID）和 `role` 字段，供后续接口鉴权使用：

```
# app/api/auth.py（核心片段）
@auth_bp.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    user = User.query.filter_by(username=data['username']).first()
    if not user or not user.check_password(data['password']):
        return error_response('用户名或密码错误', 401)
    token = create_access_token(
        identity=str(user.id),
        additional_claims={'role': user.role.value}
    )
    return success_response({'token': token, 'role': user.role.value,
                             'real_name': user.real_name})
```

需要权限控制的接口则使用自定义装饰器 `require_roles`，用法如下：

```
@attendance_bp.route('/tasks', methods=['POST'])
@require_roles('admin', 'teacher')
```

```
def create_task():
    ...
```

6.1.4 5.1.4 考勤任务 API 实现

考勤任务模块是系统的核心业务，下面展示开始和结束考勤的关键实现。开始考勤时需要记录课程总人数快照，避免后续选课变动影响统计：

```
@attendance_bp.route('/tasks/<int:task_id>/start', methods=['POST'])
@require_roles('admin', 'teacher')
def start_task(task_id):
    task = AttendanceTask.query.get_or_404(task_id)
    if task.status != TaskStatus.pending:
        return error_response('任务状态不允许此操作', 400)

    task.total_students = len(task.course.students)
    task.status = TaskStatus.running
    task.start_time = datetime.utcnow()
    db.session.commit()

    socketio.emit('task_update', {
        'task_id': task_id, 'status': 'running',
        'total_students': task.total_students, 'present_count': 0
    }, room=f'task_{task_id}')
    return success_response({'task_id': task_id, 'status': 'running'})
```

停止考勤时，自动为未出勤学生批量生成缺勤记录，确保数据完整性：

```
@attendance_bp.route('/tasks/<int:task_id>/stop', methods=['POST'])
@require_roles('admin', 'teacher')
def stop_task(task_id):
    task = AttendanceTask.query.get_or_404(task_id)
    present_ids = {r.student_id for r in task.records
                   if r.status in (RecordStatus.present, RecordStatus.unverified)}
    absent_records = [
        AttendanceRecord(task_id=task_id, student_id=s.id,
```

```
                        status=RecordStatus.absent)
        for s in task.course.students if s.id not in present_ids
    ]
    if absent_records:
        db.session.bulk_save_objects(absent_records)
    task.status = TaskStatus.finished
    task.end_time = datetime.utcnow()
    db.session.commit()
    return success_response({'absent_count': len(absent_records)})
```

6.2 5.2 WebSocket 实时识别实现

6.2.1 5.2.1 服务端帧处理逻辑

WebSocket 处理模块接收前端推送的 Base64 编码视频帧，调用 AI 推理模块进行识别，并把结果广播到对应考勤任务的 Socket.IO 房间：

```
# app/ws/handlers.py（核心片段）
@socketio.on('frame')
def on_frame(data):
    task_id = data.get('task_id')
    frame_b64 = data.get('frame', '')

    img_bytes = base64.b64decode(frame_b64.split(',')[-1])
    nparr = np.frombuffer(img_bytes, np.uint8)
    frame = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
    if frame is None:
        return

    manager = ModelManager.get_instance()
    results = manager.recognize_faces(frame)

    for result in results:
```

```
        if result['status'] in ('present', 'unverified'):
            _save_record(task_id, result)

    emit('recognized', {'results': results, 'task_id': task_id},
         room=f'task_{task_id}')
    _emit_task_stats(task_id)
```

6.2.2 5.2.2 防重复出勤处理

同一学生在一次考勤任务中可能被多次识别到，需要在写库时做幂等性保护，避免重复写入：

```
def _save_record(task_id, result):
    student_db_id = result.get('student_db_id')
    if not student_db_id or student_db_id < 0:
        return
    existing = AttendanceRecord.query.filter_by(
        task_id=task_id, student_id=student_db_id
    ).first()
    if existing and existing.status == RecordStatus.present:
        return  # 已高置信确认，直接跳过
    if existing is None:
        record = AttendanceRecord(
            task_id=task_id, student_id=student_db_id,
            status=RecordStatus[result['status']],
            confidence=result['confidence'],
            recognized_at=datetime.utcnow()
        )
        db.session.add(record)
    else:
        # 如果旧记录是 unverified 而新识别是 present，就把状态升级一下
        if result['status'] == 'present':
            existing.status = RecordStatus.present
            existing.confidence = result['confidence']
    db.session.commit()
```

6.3 5.3 AI 推理模块实现

6.3.1 5.3.1 ModelManager 单例设计

ModelManager 采用线程安全的单例模式，确保全局只加载一份模型权重，避免重复占用内存：

```
class ModelManager:
    _instance = None
    _lock = threading.Lock()

    @classmethod
    def get_instance(cls):
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:
                    cls._instance = cls._create()
        return cls._instance

    def recognize_faces(self, frame):
        boxes = self.detector.detect(frame)
        if not boxes:
            return []
        faces = self.recognizer.preprocess(frame, boxes)
        embeddings = self.recognizer.embed(faces)
        results = self.feature_db.match(embeddings, boxes)
        return results
```

6.3.2 5.3.2 特征提取：512 维 Embedding

FaceRecognizer 加载训练好的 ResNet50 权重，在推理时截断到 512 维特征层（不含最终分类头）：

```python
class FaceRecognizer:
    def __init__(self, model_type='resnet50'):
        meta = json.load(open(META_PATH))
        self.model_type = model_type
        self.model = build_model(model_type, meta['num_classes'], pretrained=False)
        self.model.load_state_dict(
            torch.load(CKPT_PATH, map_location='cpu', weights_only=True)
        )
        self.model.eval()
        self.transform = get_transforms(is_train=False)

    def embed(self, face_tensors: torch.Tensor) -> np.ndarray:
        with torch.no_grad():
            feats = _forward_embedding(self.model, self.model_type, face_tensors)
            feats = torch.nn.functional.normalize(feats, p=2, dim=1)
        return feats.cpu().numpy()
```

6.3.3 5.3.3 特征库匹配与双阈值判断

```python
class FeatureDatabase:
    HIGH_THRESHOLD = 0.75
    LOW_THRESHOLD  = 0.50

    def __init__(self):
        db_data = np.load(FEATURE_DB_PATH, allow_pickle=True).item()
        self.student_ids  = db_data['student_ids']
        self.db_ids       = db_data['db_ids']
        matrix = db_data['feature_matrix'].astype(np.float32)
        norms  = np.linalg.norm(matrix, axis=1, keepdims=True)
        self.matrix = matrix / (norms + 1e-8)  # 预先归一化，加快匹配速度

    def match(self, embeddings: np.ndarray, boxes) -> list:
        sims = embeddings @ self.matrix.T  # 批量余弦相似度
        results = []
```

```python
        for i, (sim_row, box) in enumerate(zip(sims, boxes)):
            best_idx = int(sim_row.argmax())
            best_sim = float(sim_row[best_idx])
            if best_sim >= self.HIGH_THRESHOLD:
                status = 'present'
            elif best_sim >= self.LOW_THRESHOLD:
                status = 'unverified'
            else:
                status = 'rejected'
                best_idx = -1
            results.append({
                'student_id':    self.student_ids[best_idx] if best_idx >= 0 else None,
                'student_db_id': self.db_ids[best_idx]      if best_idx >= 0 else -1,
                'status':        status,
                'confidence':    best_sim,
                'box':           box.tolist() if hasattr(box, 'tolist') else box,
            })
        return results
```

6.4 5.4 前端核心实现

6.4.1 5.4.1 全局请求封装与错误处理

前端通过 `api/request.js` 统一封装 Axios，处理 Token 注入以及 401/403 响应和错误提示：

```python
const request = axios.create({ baseURL: '/api', timeout: 10000 })

request.interceptors.request.use(config => {
  const auth = useAuthStore()
  if (auth.token) config.headers.Authorization = `Bearer ${auth.token}`
  return config
})
```

```
request.interceptors.response.use(
  res => res.data,
  err => {
    const status = err.response?.status
    if (status === 401) {
      useAuthStore().logout()
      router.push('/login')
    } else if (status === 403 && !err.config?.silent) {
      ElMessage.error('权限不足，无法执行此操作')
    } else if (!err.config?.silent) {
      ElMessage.error(err.response?.data?.message || '请求失败')
    }
    return Promise.reject(err)
  }
)
```

6.4.2 5.4.2 实时考勤组件实现

attendanceStore 负责维护 Socket.IO 连接、摄像头定时器和识别状态，下面是核心逻辑：

```
actions: {
  joinTask(taskId) {
    this.socket = io('/', { path: '/socket.io' })
    this.socket.emit('join_task', { task_id: taskId })
    this.socket.on('recognized', data => {
      data.results.forEach(r => this._mergeResult(r))
    })
    this.socket.on('task_update', data => { this.stats = data })
  },
  startCamera(taskId) {
    const video = document.getElementById('camera-video')
    navigator.mediaDevices.getUserMedia({ video: true })
      .then(stream => {
        video.srcObject = stream
```

```
        this.cameraTimer = setInterval(() => {
          const canvas = document.createElement('canvas')
          canvas.width = 320; canvas.height = 240
          canvas.getContext('2d').drawImage(video, 0, 0, 320, 240)
          const base64 = canvas.toDataURL('image/jpeg', 0.7)
          this.socket.emit('frame', { task_id: taskId, frame: base64 })
        }, 1000)  // 1fps
      })
  }
}
```

前端以 1fps 的频率抓取摄像头画面，分辨率设为 320×240 并以 JPEG 格式压缩，在保证人脸可识别的前提下大大减少了每帧数据量，降低了网络传输压力。

6.4.3 5.4.3 角色自适应菜单实现

侧边栏导航根据当前用户角色动态筛选菜单项，无权限的菜单不显示出来：

```
const menuItems = computed(() => {
  const role = auth.userInfo?.role
  return allMenuItems.filter(item =>
    !item.roles || item.roles.includes(role)
  )
})
```

此外，考勤任务列表中的操作按钮（开始考勤、监控、删除）对辅导员不可见，辅导员只能查看任务列表但不能进行操作，通过以下代码控制：

```
const canOperate = computed(() =>
  ['admin', 'teacher'].includes(auth.userInfo?.role)
)
```

6.5 5.5 本章小结

本章把主要的实现细节都过了一遍。后端以 Flask 工厂模式和 Blueprint 做模块切分，JWT 无状态鉴权处理接口安全问题；WebSocket 帧处理那块引入房间隔离和幂等保护，确保多任务并发下结果不串；AI 推理模块用 ModelManager 单例统一管理模型加载，避免重复占用内存，MTCNN 检测、ResNet50 特征提取、双阈值匹配串成一条完整流水线；前端用 Pinia 集中管理摄像头和 Socket.IO 状态，各角色的 UI 差异在这层统一处理。这些模块拼在一起，把从摄像头采帧到考勤结果写库的整个链路跑通了。

---

7 第六章系统测试与实验

7.1 6.1 功能测试

7.1.1 6.1.1 测试环境

项目	配置
操作系统	Ubuntu 22.04 LTS
CPU	x86-64，无独立 GPU
内存	4GB RAM
后端运行时	Python 3.10，Flask 3.x，eventlet
前端浏览器	Chrome 124（基于 Chromium）
数据库	MySQL 8.0

7.1.2 6.1.2 功能测试用例

采用黑盒测试方法，按模块设计测试用例，验证系统核心功能的正确性：

用户认证模块

用例编号	测试场景	输入	期望结果
TC-01	正常登录	合法用户名/密码	返回 JWT Token 和用户角色
TC-02	密码错误	正确用户名 + 错误密码	返回 401，提示 “用户名或密码错
TC-03	Token 过期	携带过期 Token 请求	返回 401，前端自动跳转登录页
TC-04	辅导员访问无权限接口	辅导员 Token + POST /tasks	返回 403，弹出 “权限不足” 提示

考勤任务模块（核心）

用例编号	测试场景	期望结果	实际结果
TC-05	教师创建考勤任务	任务创建成功，状态为 pending	通过
TC-06	开始考勤	状态变为 running，记录 total_students 快照	通过
TC-07	重复开始已运行任务	返回 400，提示 “任务状态不允许此操作”	通过
TC-08	WebSocket 帧识别	收到 frame 事件后推送 recognized 结果	通过
TC-09	同一学生重复识别	第二次识别不重复写库（幂等保护）	通过
TC-10	结束考勤	未出勤学生自动生成 absent 记录	通过
TC-11	辅导员查看任务列表	列表可见，操作按钮不显示	通过
TC-12	删除运行中任务	返回 400，阻止删除	通过

学生档案与人脸模块

用例编号	测试场景	期望结果	实际结果
TC-13	Excel 批量导入学生	导入成功，重复学号跳过，返回统计	通过
TC-14	人脸图片采集与存储	MTCNN 检测成功后裁剪存储，显示图片数	通过
TC-15	模型训练触发	异步启动训练，进度可查询	通过

上述 15 个测试用例全部通过，说明系统核心功能的实现是正确的。

7.1.3 6.1.3 边界条件测试

除了正常功能测试，还对以下边界条件进行了验证：

- **空特征库**：模型未训练时发送视频帧，系统返回空识别结果，不会崩溃；

- **无人脸帧**：摄像头画面中没有人脸时，MTCNN 返回空框列表，推送空 results 数组，没有异常；
- **学生未选课**：未加入任何课程的学生不会出现在考勤统计中；
- **并发考勤**：同时开启两个不同课程的考勤任务，各自的 WebSocket 房间互不干扰，识别结果路由正确。

从以上测试可以看出，系统在各种边界情况下都能够正常运行，健壮性比较好。

7.2 6.2 AI 模型实验

7.2.1 6.2.1 实验数据集

本实验使用 LFW（Labeled Faces in the Wild）数据集 [7]。LFW 是人脸识别领域中一个经典的基准数据集，由 Huang 等人在 2008 年发布，包含 5,749 人共 13,233 张人脸图片，这些图片来源于互联网新闻照片，在光照条件、人脸姿态、背景环境等方面都有比较大的变化，具有一定的挑战性。

考虑到实际课堂考勤场景中班级规模通常在 30 到 60 人之间，本文选取了图片数量最多的前 50 人作为实验子集（每人图片数在 53 到 530 张之间），用来模拟真实的班级规模。数据集按 70%/20%/10% 的比例划分为训练、验证、测试三个不重叠的子集，其中**测试集在整个实验期间完全封存，只在最终评估时使用一次**。

划分	图片数	类别数	每类平均张数
训练集	1,920	50	38.4
验证集	549	50	11.0
测试集	320	50	6.4

需要说明的是，测试集每类平均只有 6.4 张图片，这是由 LFW 数据集总量有限决定的，这一特点对评估指标有一些重要影响，在后续分析中会详细讨论。

7.2.2 6.2.2 训练配置

配置项	值
骨干网络	ResNet50（ImageNet 预训练）
分类头	FC(2048→512)→ReLU→Dropout(0.3)→FC(512→50)
训练设备	CPU（x86-64，4GB RAM）
总训练轮数	30 epochs
Phase 1（Epochs 1~10）	冻结主干，Adam，$lr = 10^{-3}$，固定学习率
Phase 2（Epochs 11~30）	解冻全部，SGD+Momentum($m = 0.9$)，$lr = 10^{-4}$，余弦退火
Batch Size	16
数据增强	随机裁剪（256→224）、水平翻转、颜色抖动、随机旋转 ±15°
最优模型保存依据	验证集 Top-1 准确率最高

关于 Phase 2 采用 SGD 而不是 Adam 的原因，前面在技术综述部分已经提到了，这里再简单说一下：SGD 在 CPU 环境下内存占用约为 Adam 的三分之一，在本实验的 4GB 内存环境下这一点特别重要；此外，余璀璨和李慧斌 [12] 等研究也发现 SGD+Momentum 在微调场景下往往有更好的泛化表现。

7.2.3 6.2.3 训练过程分析

训练过程呈现出比较典型的两阶段收敛特征，从这个过程中可以看出两阶段策略的效果还是比较明显的：

Phase 1（Epochs 1~10，冻结阶段）：验证集准确率从初始约 13% 快速上升到约 45%，这说明 ImageNet 预训练特征对人脸识别任务是有效的，只需要训练参数量远小于主干的分类头，就可以在 10 轮内建立起有效的分类能力。

Phase 2（Epochs 11~30，全局微调阶段）：解冻全部参数后，学习率降到 10^{-4}，验证准确率持续稳步提升，Epoch 16 达到 78.61%，Epoch 20 达到 83.90%，最终在 **Epoch 26 达到峰值 86.30%**，之后轻微波动直到训练结束。余弦退火调度使学习率平滑衰减，比较有效地防止了末期的过拟合现象。

Epoch	val_acc	备注
1	13.11%	Phase 1 开始
5	30.24%	分类头快速收敛
10	45.40%	Phase 1 结束

Epoch	val_acc	备注
11	~50%+	Phase 2 开始（解冻全部层）
16	78.61%	Phase 2 稳步提升
20	83.90%	
26	**86.30%**	最优检查点（保存）
30	85.02%	轻微回落，训练结束

7.2.4 6.2.4 测试集评估结果

使用封存的测试集对 Epoch 26 的最优检查点进行评估，从实验中可以得到如下结果：

指标	实验结果	目标值	是否达标
Top-1 准确率	**81.87%**	⩾ 85%	接近但未达
Precision（加权）	79.93%	⩾ 0.85	接近但未达
Recall（加权）	81.87%	⩾ 0.85	接近但未达
F1 Score（加权）	79.16%	⩾ 0.85	接近但未达
AUC-ROC（macro）	**99.30%**	⩾ 0.90	大幅达标
FAR 误识率（加权）	**1.17%**	⩽ 5%	达标
FRR 拒识率（加权）	18.12%	⩽ 10%	未达标
推理延迟（ms/帧）	**176.48ms**	⩽ 200ms	达标

宏平均（仅供参考）：Precision=75.83%，Recall=73.26%，F1=72.16%。

针对上述数字，有几点需要结合数据集特点解释。

验证集与测试集的准确率差异（86.30% vs 81.87%）是数据集分割比例决定的，不是过拟合。验证集每类约 11 张，测试集每类平均只有 6.4 张，某些类别甚至只有 3 到 5 张。Top-1 准确率对每张样本等权计算，少量低质量图片（极端角度、运动模糊）在小样本类别里占比偏高，直接压低了测试集数字。换句话说，模型没有问题，是测试集本身更难。

AUC-ROC 达到 99.30% 是这组结果里最值得关注的数据。AUC-ROC 是阈值无关的指标，衡量的是模型把“这个人”和“其他所有人”分开的能力。99.30% 意味着模型的特征空间分布质量很高，在任意阈值下都能维持强辨别力。这比 Top-1 准确率更能反映模型在实际部署中的可靠性。

FAR 只有 1.17%，远低于 5% 的目标。误识率低说明系统不太会把陌生人识别成已注册学生，防冒名顶替这块表现不错。

FRR 达到 18.12% 则是不理想的地方，没能满足 ≤10% 的目标。背后有几个叠加因素：HIGH_THRESHOLD 设为 0.75 相对严格；LFW 图片来源于新闻媒体，光照和姿态变化远超受控摄像头场景；特征库用的是平均向量，单个向量不一定能覆盖同一人的所有外观变化。工程上的应对是引入 0.50~0.75 之间的 “待核实” 状态，让大量本会被直接拒识的情况转为人工复核，避免了把真实学生误标为缺勤。

推理延迟 176.48ms 包含了 MTCNN 检测、ResNet50 特征提取、余弦匹配的全部耗时，在纯 CPU 环境下满足 ≤200ms 目标，说明系统在资源受限的普通服务器上可以实际运行。

最后做个简短的汇总：AUC-ROC（99.30%）、FAR（1.17%）、推理延迟（176.48ms）和验证集 Top-1（86.30%）四项全部达标；测试集 Top-1（81.87%）受测试集样本量偏小影响略低于目标，不构成模型本身的问题；FRR 偏高通过双阈值机制在工程层面做了补救，没有简单地把漏识等同于缺勤。

7.3 6.3 VGG16 与 ResNet50 架构对比分析

7.3.1 6.3.1 架构参数对比

为了支撑模型选型的决策，本节从架构参数层面对两个模型进行系统性对比：

对比维度	VGG16	ResNet50
总参数量	**1.38 亿**	**2,550 万**
主干特征层参数	约 9,200 万（卷积层）	约 2,320 万（残差块）
分类头参数（本文设计）	约 1.08 亿（25088→4096→512→C）	约 107 万（2048→512→C）
模型权重文件大小（本文训练）	**457 MB**	**95 MB**
输入分类头维度	25088（7×7×512）	2048
最大单层内存（25088→4096 FC）	**~412 MB**（fp32）	—
网络深度	16 层（13 卷积 +3 全连接）	50 层（含残差块）

从这些数字可以看出，VGG16 的参数量是 ResNet50 的约 5.4 倍，而且大部分参数都集中

在全连接层，这给内存管理带来了很大压力。

7.3.2 6.3.2 训练过程对比

两个模型在相同数据集（LFW Top-50，train/val/test=1920/549/320）和相同两阶段训练策略下的训练过程差异还是相当显著的：

训练维度	ResNet50	VGG16
初始运行内存	4GB RAM，2GB Swap	4GB RAM，2GB Swap（GAP 改造后内存大
batch_size	16	32
每轮训练时间	约 12 分钟	Phase 1 约 16 分钟 / Phase 2 约 40 分钟
Phase 1 末（Ep.10）val_acc	**45.40%**	43.34%
最优 val_acc	**86.30%**（Ep.26）	**74.67%**（Ep.26，完整 30 轮）
总训练时长	约 90 分钟	约 16 小时（实测，完整 30 轮）

VGG16 采用 GAP 架构后，分类头参数量由原来的约 1.08 亿骤降至约 26 万，batch_size 得以从 4 提升至 32，Phase 1 末验证准确率达到 43.34%，与 ResNet50（45.40%）已基本持平，梯度噪声问题基本得到解决。但 VGG16 主干（约 1.38 亿参数）在 Phase 2 解冻全局微调时每轮耗时约 40 分钟，是 ResNet50（约 12 分钟）的 3 倍以上，这一差距来自卷积主干浮点计算量的结构性差异，无法通过参数调整消除。

7.3.3 6.3.3 测试集性能指标实测对比

对两个模型当前最优检查点在封存测试集上的完整评估结果如下：

指标	ResNet50（Ep.26）	VGG16-GAP（Ep.26）	目标
Top-1 准确率	**81.87%**	71.25%	⩾85%
Precision（加权）	**79.93%**	71.35%	⩾85%
Recall（加权）	**81.87%**	71.25%	⩾85%
F1 Score（加权）	**79.16%**	69.59%	⩾85%
AUC-ROC（macro）	**99.30%**	98.22%	⩾90%
FAR 误识率	**1.17%**	0.99%	⩽5%
FRR 拒识率	**18.12%**	28.75%	⩽10%

指标	ResNet50（Ep.26）	VGG16-GAP（Ep.26）	目标
推理延迟（CPU）	**176.48ms**	405.67ms	≤200ms

两个模型均已完整训练至 30 轮（均取 Ep.26 最优检查点）。VGG16 采用 GAP 改造后，精度指标相比旧 FC 头版本大幅提升，AUC-ROC（98.22%）与 ResNet50（99.30%）差距已收窄至 1 个百分点，FAR（0.99%）甚至略优于 ResNet50（1.17%）。然而**推理延迟（405.67ms）仍是 200ms 目标的 2 倍以上**，这一限制来源于 VGG16 卷积主干（1.38 亿参数）带来的浮点运算量，是结构性约束，不会因 GAP 改造或继续训练而改变。FRR 偏高（28.75%）同样受限于主干表达能力，与 ResNet50（18.12%）存在明显差距。

7.3.4 6.3.4 模型选型结论

8 项指标 ResNet50 全部胜出，但采用 GAP 架构后 VGG16 在精度上与 ResNet50 的差距已大幅收窄。

精度层面，VGG16-GAP（Top-1 71.25%，AUC-ROC 98.22%）与 ResNet50（Top-1 81.87%，AUC-ROC 99.30%）相比差距明显缩小，说明 FC 头冗余参数是旧版 VGG16 表现差的主要原因之一，而非卷积主干本身的问题。但 **推理延迟依然是结构性瓶颈**：VGG16 的 405ms 是 ResNet50（176ms）的 2.3 倍，超出 200ms 实时性目标一倍，这一差距来自卷积主干参数量（1.38 亿 vs 2550 万）带来的固定浮点计算负担，GAP 改造对推理延迟几乎没有帮助，因为延迟的瓶颈在特征提取而非分类头。权重文件体积（228MB vs 98MB，GAP 版已较 FC 版的 528MB 大幅减小）仍是 ResNet50 的两倍多，这也是部署时需要考量的实际约束。

训练时间方面，GAP 改造虽然解决了内存和收敛问题，但 Phase 2 全局微调每轮耗时约 40 分钟（ResNet50 约 12 分钟），总训练时长约 16 小时，调参成本仍然偏高。

在本系统 CPU 部署场景下，ResNet50 是唯一可行的选择。这也与屈东东等 [13] 的研究结论一致：在资源受限的 CPU 场景中，控制模型参数量对推理延迟的影响远比精调超参数更为关键。

7.4 6.4 系统性能测试

7.4.1 6.4.1 接口响应时间

在本地开发环境（4GB CPU 服务器）下，对核心接口进行响应时间测量，结果如下：

接口	平均响应时间	说明
POST /api/auth/login	85ms	含 bcrypt 密码验证（cost=12）
GET /api/attendance/tasks	45ms	分页查询，20 条记录
GET /api/reports/dashboard	120ms	含多表聚合统计
WebSocket 帧识别（端到端）	176.48ms	含 MTCNN+ResNet50+ 匹配
POST /api/attendance/tasks/:id/stop	230ms	含批量生成缺勤记录

从测试结果来看，除了 WebSocket 帧识别接口受 AI 推理耗时影响较长（176.48ms）之外，其他接口响应时间都比较快，数据库查询类接口基本在 50ms 到 120ms 范围内，用户使用体验应该是可以接受的。

7.4.2 6.4.2 WebSocket 并发测试

使用 WebSocket 客户端模拟两个考勤任务同时发送视频帧，验证 Socket.IO 房间隔离机制的正确性：

- **Task A**（课程 1）：识别结果只广播到 `room=task_1`，Task B 的客户端收不到；
- **Task B**（课程 2）：识别结果只广播到 `room=task_2`，Task A 的客户端收不到；
- 两个任务并发运行时，单帧识别延迟没有发生显著增加（平均延迟差小于 20ms）。

测试结果表明，Socket.IO 的房间隔离机制运行正常，不同考勤任务之间确实互不干扰，系统可以支持多教室同时进行考勤的实际使用场景。

7.5 6.5 本章小结

本章的实验结果可以用几句话概括。功能测试 15 个用例全部通过，边界条件（大文件上传、人脸未注册、并发考勤等）也都验证了正常。AI 模型层面，ResNet50 在 LFW Top-50 上跑出 86.30% 验证集准确率、99.30% AUC-ROC、1.17% FAR、176.48ms 推理延迟，关键

指标均达标。VGG16 经 GAP 架构改造后完整训练 30 轮，精度大幅提升（验证集 74.67%，测试集 Top-1 71.25%，AUC-ROC 98.22%），但 CPU 推理延迟（405ms）仍超出实时性目标一倍，为结构性约束；ResNet50 在精度和延迟两项上均优于 VGG16，选型有充分的实验支撑。

---

8 第七章总结与展望

8.1 7.1 工作总结

本文做的事情说起来并不复杂：设计并实现了一套基于卷积神经网络的校园课堂智能考勤系统。但落地过程中遇到的问题远比预想的多，这里按实际完成的工作做个梳理。

系统层面，采用 Vue 3 + Flask 的前后端分离架构，以 JWT 为基础实现了管理员、教师、辅导员、学生四种角色各自独立的权限边界。前端 11 个功能页面，后端 6 个 Blueprint 模块、约 30 个 REST 接口和 4 类 WebSocket 事件，把学生信息管理、人脸采集、实时识别、记录查询、报表导出这几块完整串联起来。考勤这件事本身的业务逻辑并不算复杂，难的是把实时视频流、AI 推理和多端通知这三件事同时做稳——最终用 Flask + SocketIO 的方案解决了这个问题。

模型层面，最关键的工作是确定并验证了两阶段迁移学习策略。Phase 1 冻结主干、用 Adam（lr=10^{-3}）快速收敛分类头；Phase 2 解冻全部参数、改用 SGD + Momentum 配合余弦退火做全局微调。在 LFW Top-50 子集上，ResNet50 跑出了 86.30% 的验证集 Top-1 准确率、99.30% 的 AUC-ROC、1.17% 的 FAR，CPU 单帧推理延迟 176.48ms，四项指标都满足或超出了任务书要求。说实话，AUC-ROC 能到 99.30% 还挺出乎意料的，主要得益于 Phase 2 解冻之后余弦退火对学习率的精细控制。

推理流水线这边，设计了 “MTCNN 检测 → ResNet50 特征提取 → L2 归一化 → 余弦相似度匹配 → 双阈值判断” 的完整链路。双阈值（高 0.75 / 低 0.50）引入了 “待核实” 中间状态，在误识率和拒识率之间找到了一个相对实用的平衡点，避免了单阈值下非此即彼的粗暴判断。

对比实验方面，在相同数据集和训练设置下完整跑了 VGG16-GAP 和 ResNet50 两组实验（均 30 轮）。结论很明确：ResNet50 在 8 项指标上全部优于 VGG16，尽管 GAP 改造后

VGG16 精度已大幅提升（Top-1 71.25%，AUC-ROC 98.22%），但推理延迟差距是结构性的——VGG16 的 405ms 超出实时性目标整整一倍，直接排除了它在本系统 CPU 场景下的部署可能性。这组对比数据从实验层面把模型选型的依据说清楚了：约束不在训练，而在参数量决定的浮点计算量。

8.2 7.2 不足与局限

有必要把系统当前的局限性讲清楚，这样下一步改进才有方向。

最明显的问题是 FRR 偏高（18.12%），超出了任务书设定的 10% 目标。这个问题的根源在 LFW 数据集本身——样本多样性大，光照、角度、遮挡的变化范围远超课堂真实场景，导致模型在一些难样本上拒识偏多。“待核实” 状态和人工复核机制在工程上缓解了这个问题，但如果场景是几百人的大班、要求快速无感点名，18% 的漏识率依然会给教师带来明显的复核负担。

另一个设计上的遗憾是单帧独立识别。当前流水线对每帧做独立判断，同一个学生在视频流里连续出现十几帧，会被反复识别十几次，这部分计算是纯粹的浪费。更重要的是，单帧结果容易受运动模糊或短暂遮挡的影响而跳变，如果引入多帧投票之类的时序融合就能平滑这种噪声，但这次没来得及做。

特征库的静态设计也是个实际问题。每次新增学生，都必须重新训练模型并重建整个特征向量库，不存在 “加一个人就更新一个原型” 的增量机制。班级人员变动越频繁，这个问题就越麻烦，维护成本会随规模线性增长。

最后，系统全程在开发环境单节点上运行，从未做过高并发压测，也没有按生产标准加固安全（缺少 HTTPS 配置、没有请求频率限制、SQL 层面的防护也只是框架默认的基础配置）。这意味着当前的性能数据只能说明功能可行，不能代表生产容量。

8.3 7.3 未来展望

结合上述局限性，有几个方向我认为是值得继续做的，按优先级大致排一下。

损失函数层面，用 ArcFace [4] 替换当前的 Softmax 应该是性价比最高的改进。ArcFace 通过加性角度间隔直接约束特征空间，让同类样本更聚拢、不同类样本更分散，不需要改动网络结构，只需替换损失函数和调整超参。论文中验证 FRR 偏高主要源于特征边界模糊，ArcFace 正好针对这一点——文献 [4] 已经在 LFW 等多个基准上验证了其对 FRR 的改善效果，改造成本也相对可控。

时序融合层面，最直接的方案是在现有流水线上叠加一个滑动窗口投票层：维护最近 N 帧的识别结果，只有当多数帧一致指向同一人时才触发出勤写入。这不需要改动底层模型，只是在后处理阶段加一层缓冲逻辑，实现起来并不复杂，但对单帧噪声的抑制效果应该很明显。

增量学习层面，思路是引入原型网络（Prototypical Networks）或持续学习框架，让系统在新增学生时只更新对应的特征原型，而不需要全量重新训练。这个方向改动幅度最大，但对实际运维最有价值——班级人员调整是高频操作，如果每次都要重跑训练，在真实学校场景下几乎不可持续。

部署层面，如果要在教室端摄像头上本地跑推理，ResNet50 的体量和 CPU 延迟都偏大。通过 INT8 量化或知识蒸馏将其压缩到 MobileNetV3 量级，延迟可以降到 50ms 以下，边缘设备上部署才现实。当然，量化本身会带来精度损失，这里有个精度与延迟的权衡需要在实际硬件上实测确认。

此外还有一个往往被忽略但不能回避的问题：合规性。人脸数据在《个人信息保护法》下属于敏感个人信息，采集前需要取得明确的书面同意，存储时只应保留特征向量而非原始图片，访问行为也需要留审计日志。这些不是可选项，而是真实部署时的法律底线。这次系统实现里这块是欠缺的，后续要补上。

---

9 参考文献

[1] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). Las Vegas, 2016: 770-778. https://arxiv.org/abs/1512.03385

[2] Simonyan K, Zisserman A. Very deep convolutional networks for large-scale image recognition[C]//International Conference on Learning Representations

(ICLR). San Diego, 2015. https://arxiv.org/abs/1409.1556

[3] Zhang K, Zhang Z, Li Z, et al. Joint face detection and alignment using multitask cascaded convolutional networks[J]. IEEE Signal Processing Letters, 2016, 23(10): 1499-1503. https://ieeexplore.ieee.org/document/7553523/ DOI: 10.1109/LSP.2016.2603342

[4] Deng J, Guo J, Xue N, et al. ArcFace: Additive angular margin loss for deep face recognition[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2019: 4690-4699. https://arxiv.org/abs/1801.07698

[5] Schroff F, Kalenichenko D, Philbin J. FaceNet: A unified embedding for face recognition and clustering[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). Boston, 2015: 815-823. https://arxiv.org/abs/1503.03832

[6] Taigman Y, Yang M, Ranzato M, et al. DeepFace: Closing the gap to human-level performance in face verification[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). Columbus, 2014: 1701-1708. https://openaccess.thecvf.com/content_cvpr_2014/html/Taigman_DeepFace_Closing_the_201

[7] Huang G B, Mattar M, Berg T, et al. Labeled faces in the wild: A database for studying face recognition in unconstrained environments[C]//Workshop on Faces in Real-Life Images: Detection, Alignment, and Recognition. Marseille, 2008. https://vis-www.cs.umass.edu/lfw/

[8] Pan S J, Yang Q. A survey on transfer learning[J]. IEEE Transactions on Knowledge and Data Engineering, 2010, 22(10): 1345-1359. https://ieeexplore.ieee.org/document/5288 DOI: 10.1109/TKDE.2009.191

[9] Ioffe S, Szegedy C. Batch normalization: Accelerating deep network training by reducing internal covariate shift[C]//Proceedings of the 32nd International Conference on Machine Learning (ICML). Lille, 2015: 448-456. https://arxiv.org/abs/1502.03167

[10] LeCun Y, Bottou L, Bengio Y, et al. Gradient-based learning applied to document recognition[J]. Proceedings of the IEEE, 1998, 86(11): 2278-2324. https://ieeexplore.ieee.org/document/726791

[11] Turk M, Pentland A. Eigenfaces for recognition[J]. Journal of Cognitive Neuroscience, 1991, 3(1): 71-86. https://direct.mit.edu/jocn/article/3/1/71/3025/Eigenfaces-for-Recognition DOI: 10.1162/jocn.1991.3.1.71

[12] 余璀璨, 李慧斌. 基于深度学习的人脸识别方法综述 [J]. 工程数学学报, 2021, 38(04): 451-469. https://www.cnki.com.cn/Article/CJFDTOTAL-GCSX202104001.htm

[13] 屈东东, 贺利乐, 何林. 改进的轻量化人脸识别算法 [J]. 智能系统学报, 2023, 18(03): 544-551. https://html.rhhz.net/tis/html/202111051.htm DOI: 10.11992/tis.202111051

[14] Song J, Kim J, Nang J. Face de-identification using convolutional neural network (CNN) models for visual-copy detection[J]. Applied Sciences, 2024, 14(5): 1771. https://www.mdpi.com/2076-3417/14/5/1771 DOI: 10.3390/app14051771

[15] 唐琳. 基于人脸识别技术的学生课堂考勤管理系统的设计与实现 [J]. 数字技术与应用, 2023, 41(09): 208-210. DOI: 10.19695/j.cnki.cn12-1369.2023.09.65

[16] 李国平，李小纳. 基于人脸识别的智慧教室学生考勤系统设计 [J]. 信息与电脑(理论版)，2023，35(16): 130-132. https://www.cnki.com.cn/Article/CJFDTOTAL-XXDN202316040.htm